Exercise 1 Hadoop 集群的安装与配置

1 实验目的

1) 掌握 Linux 虚拟机的安装方法。
2) 掌握一些常用的 Linux 命令。
3) 掌握 Hadoop 的分布式安装方法。
4) 掌握 Hadoop 的常用操作。

2 实验平台

1) 操作系统：Windows 操作系统或者 Ubuntu 操作系统（推荐）。
2) 虚拟机软件：VMWare 或者 VirtualBox（开源免费体积小）。下文将以 VMWare 为例进行讲解。读者可以在 Windows 操作系统上安装 VMWare 软件，然后在 VMWare 上安装并且运行 Linux 操作系统。本次实验默认的 Linux 发行版为 Ubuntu Kylin 16.04 LTS。Ubuntu Kylin 较新的版本是 22.04 LTS，可根据计算机性能进行选择。但是，在实际使用过程中发现，新版本对计算机的资源消耗较多，在使用虚拟机方式安装时，系统运行起来速度较慢。因此，本实验选择较低的版本 Ubuntu Kylin 16.04 LTS，这个版本不仅降低了对计算机配置的要求，也可以保证大数据各种软件的顺利安装和运行，帮助同学们很好完成本书的各个实验。
3) Hadoop 版本：3.3.5。

注：

1) 建议创建多台虚拟机搭建分布式集群系统；
2) 对于个人计算机性能受限无法创建虚拟机的情况，可使用 Windows 平台上的伪分布式方案。

3 实验内容

3.1 安装 Linux 虚拟机

如果读者正在使用 Linux 操作系统，可以跳过本步，不需要下载相关软件，也不需要安装 Linux 虚拟机；如果读者正在使用 Windows 操作系统，则需要在 Windows 操作系统上安装 Linux 虚拟机，这里就需要下载 VMWare 软件和 Ubuntu 16.04 镜像文件。VMWare 软件的官方下载地址为：https://www.vmware.com/cn/products/workstation-pro.html。Ubuntu 16.04 的镜像文件的官方下载地址为：https://www.ubuntu.org.cn/download/ubuntu-kylin。注：相关软件可直接从学院 ftp 服务器上本课程所在目录中下载（ftp://10.13.0.10）。

首先，在 Windows 系统上安装虚拟机软件 VMWare；其次，在虚拟机软件 VMWare 上安装 Ubuntu 16.04 操作系统。

课程 FTP 中提供了 Windows 平台下的 VMWare Workstation17，Mac 系统需自行到官网下载（图 1 所示）。

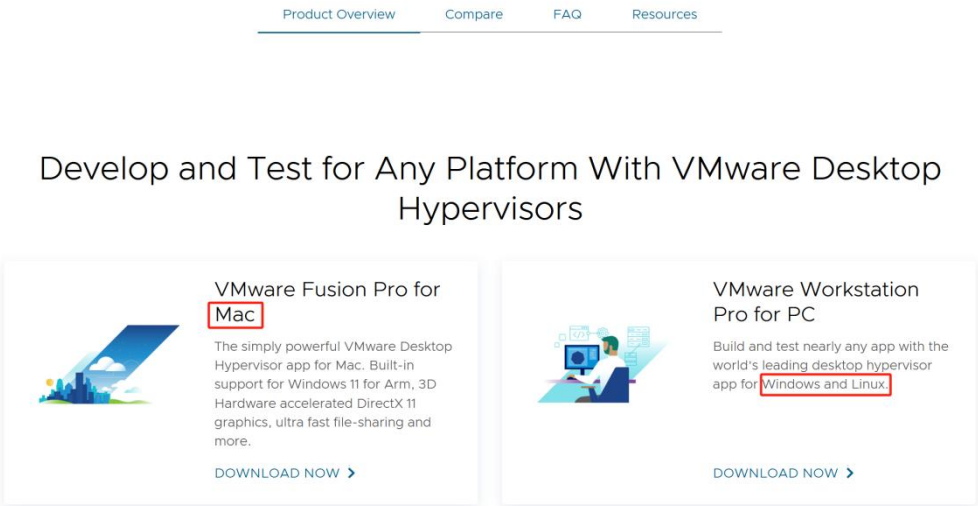


图 1. VMWare 下载页面

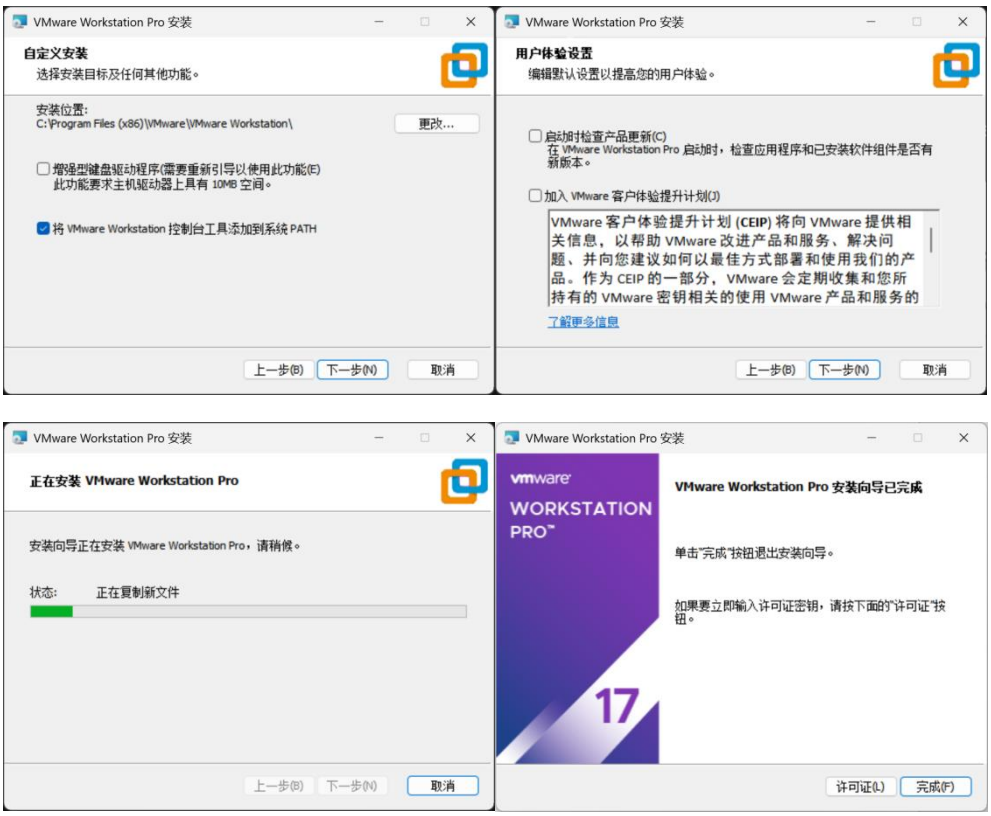


图 2. VMware 安装过程

在 VMware 安装完成之前需要输入所购买的产品序列号（图 2）。之后便可以启动 VMware 创建 Ubuntu 虚拟机了（图 3）。

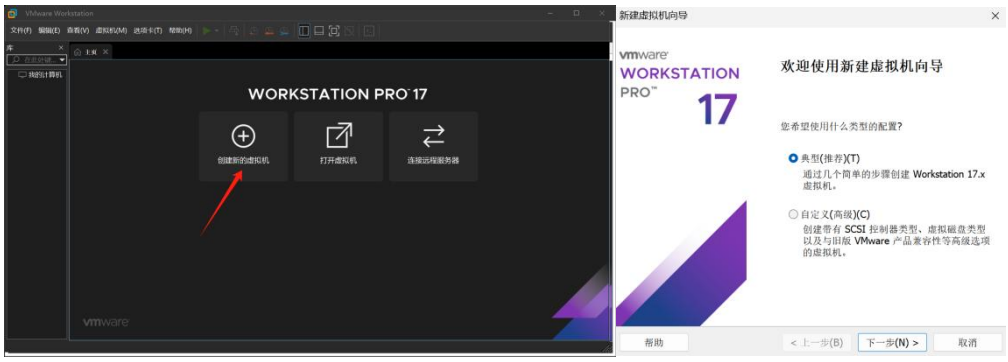


图 3. 创建虚拟机

在弹出的新建虚拟机向导中选择 Ubuntu 安装光盘 iso 镜像文件（可直接从 FTP 下载），点击下一步，进行安装配置。

图 4. 加载 Ubuntu 安装光盘镜像文件

图 5. 加载 Ubuntu 安装光盘镜像文件

进行账户密码和虚拟机名称配置。此处，全名和用户名可以相同，**全名**在登录界面显示，**用户名**在终端命令行窗口中显示。相关名称根据需要设置，在后续使用过程中还可以修改。

之后需要对虚拟机的硬件资源进行配置。首先如图 6 所示，设置硬盘大小，这里根据后续软件安装需要设置为 100GB，存储为单个文件。该虚拟硬盘文件的大小是随着使用逐步增大的，不会立即占用主机的上述硬盘空间。

图 6. 虚拟机硬件设置

除了硬盘大小之外，内存、CPU、网络适配器、显示器等均可配置。相关设置**应根据自身主机计算机资源而定**，虚拟机内存设置过大会拖慢电脑运行速度。此处，将内存设置为 8GB（一般最大不超过主机内存的一半）。所有参数在虚拟机关机时均可修改，后续可以根据实际使用情况进行调整（如图 8 所示）。

图 7. 其他各项硬件的具体配置

图 8. 其他各项硬件的具体配置

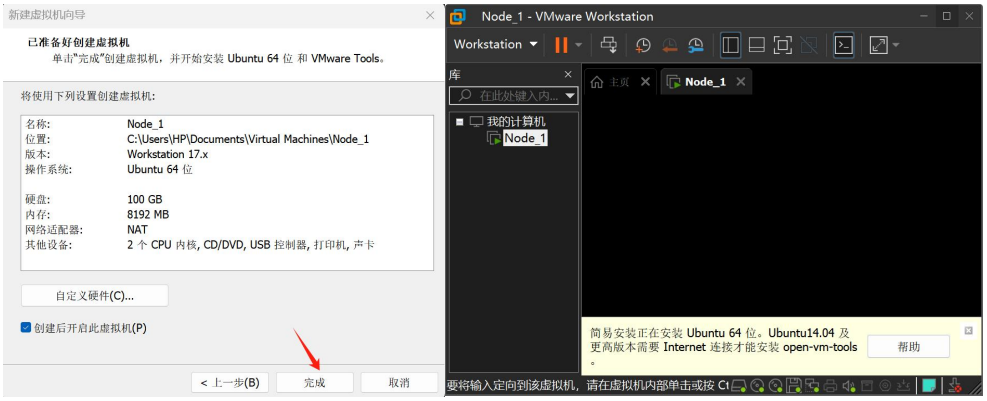


图 9. 开启虚拟机进行系统安装

设置完虚拟机的各项参数之后便可以启动虚拟机，虚拟机启动过程中检测到虚拟光驱中的 Ubuntu 操作系统安装光盘镜像，随即启动引导程序进行安装。

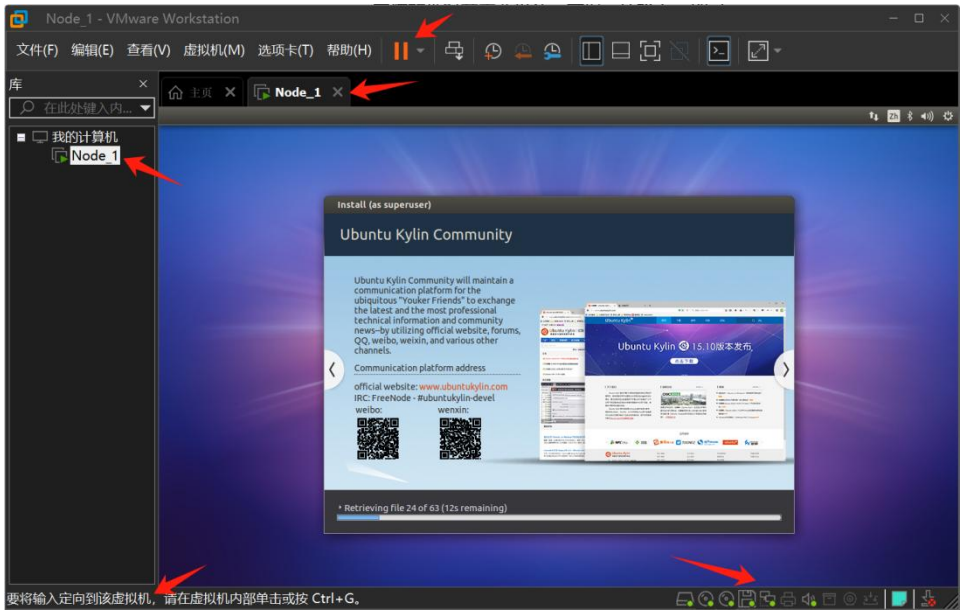


图 10. VMWare 窗口

在系统的安装过程中，我们可以观察 VMWare 软件窗口中各个功能模块的作用，包括左侧的虚拟机列表，左下角的状态提示信息，右下角的虚拟机硬件资源配置栏等（可以看到硬盘、光驱、USB 等虚拟硬件设备）。

系统安装完成后便可以进入登录界面。按照同样的方法我们可以创建多台虚拟机用于搭建分布式集群。

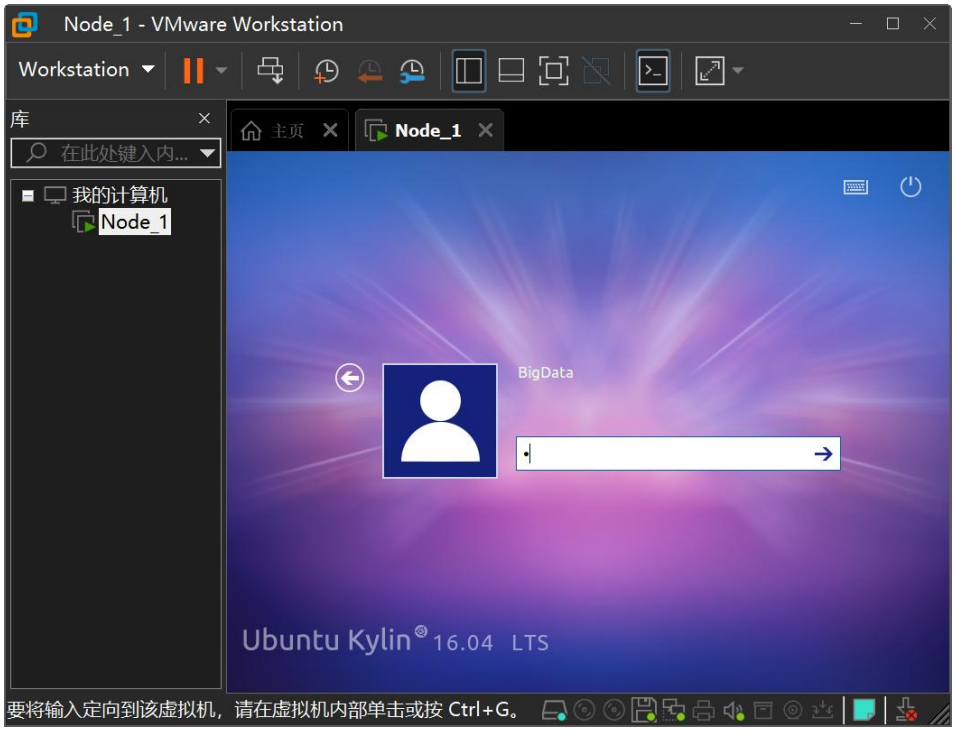
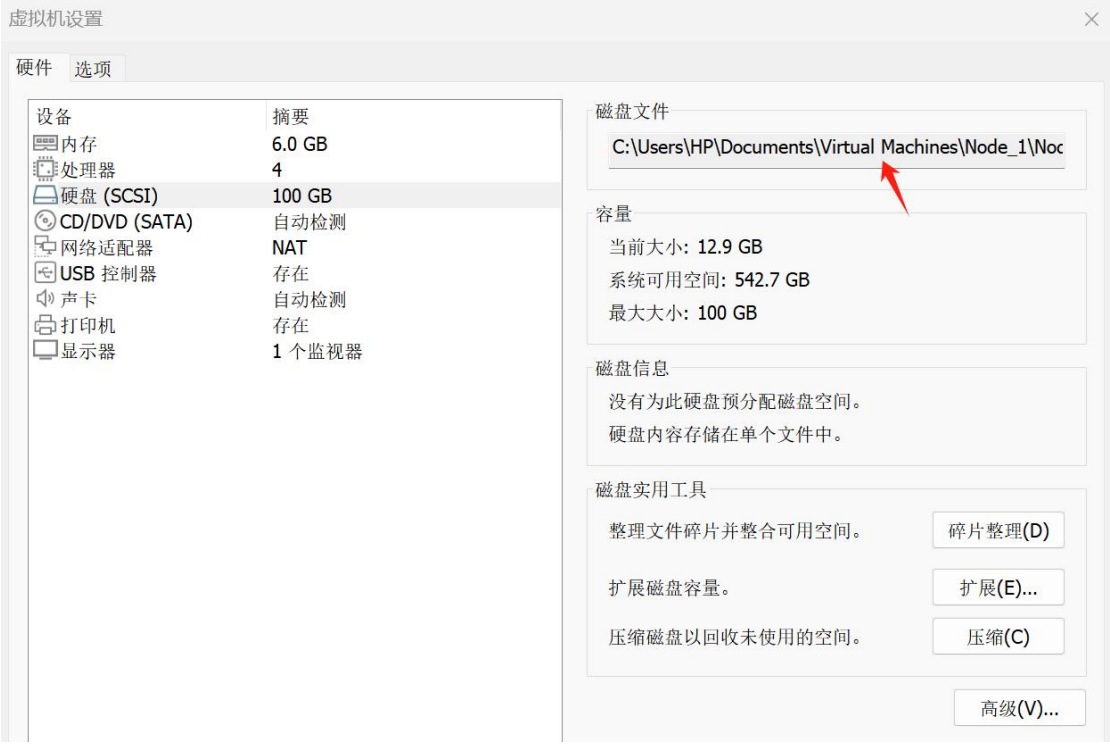


图 11. Ubuntu 用户登录窗口

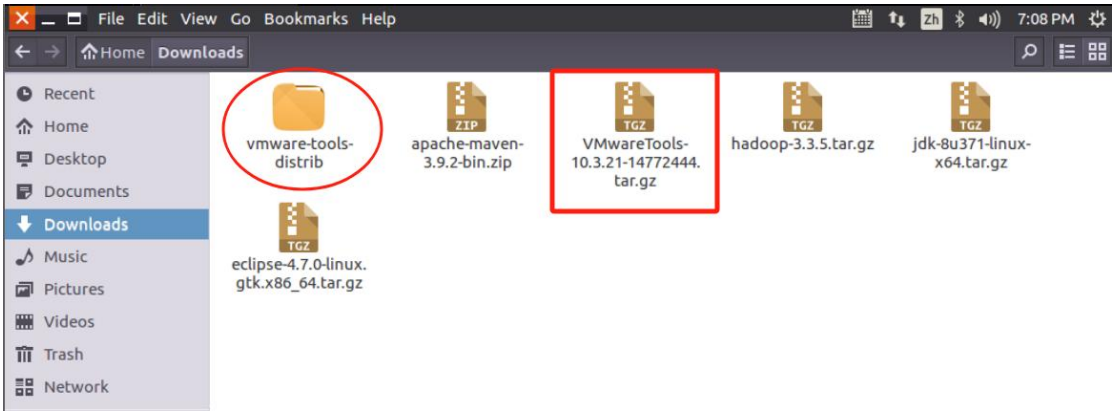


在虚拟机设置界面->硬件->硬盘选项卡中可以观察到虚拟机具体存放位置及当前实际硬盘占用大小。

安装完 Ubuntu 系统之后我们还可以继续安装 VMware **增强工具**，安装之后能够提供虚拟机显示分辨率修改、虚拟机与主机间的文件拷贝及剪切板共享等诸多便捷功能。

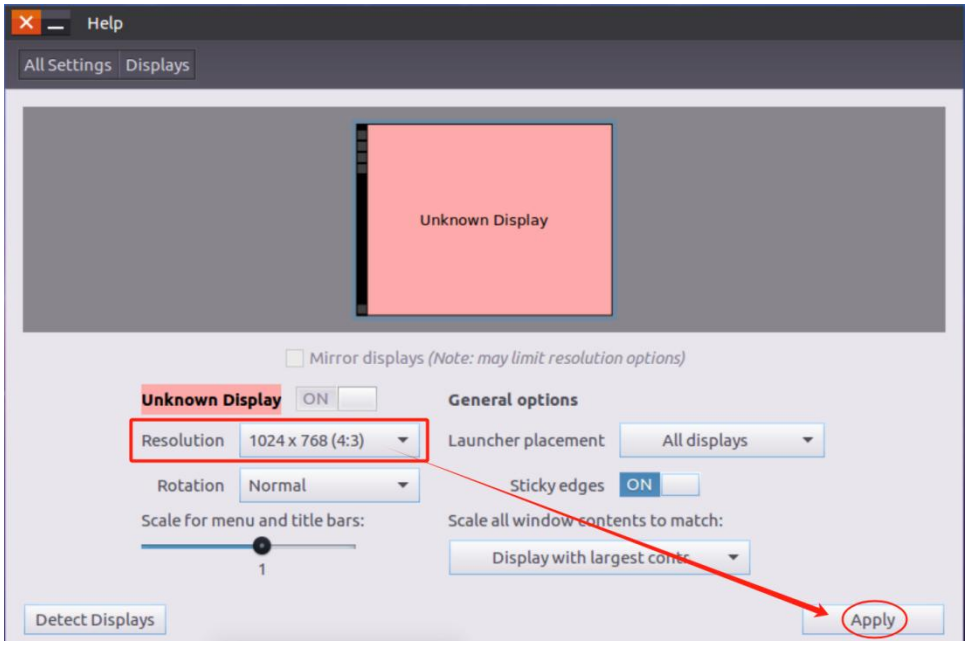
从 ftp 上下载 VMwareTools-10.3.21-14772444.tar.gz 到 Ubuntu 的 Downloads 目录下。通过如下命令解压

```
cd ~/Downloads
sudo tar -zxvf ./VMwareTools-10.3.21-14772444.tar.gz -C ./      #解压增强工具
$ cd vmware-tools-distrib                                       #进入解压目录
$ sudo ./vmware-install.pl                                      #开始安装
```

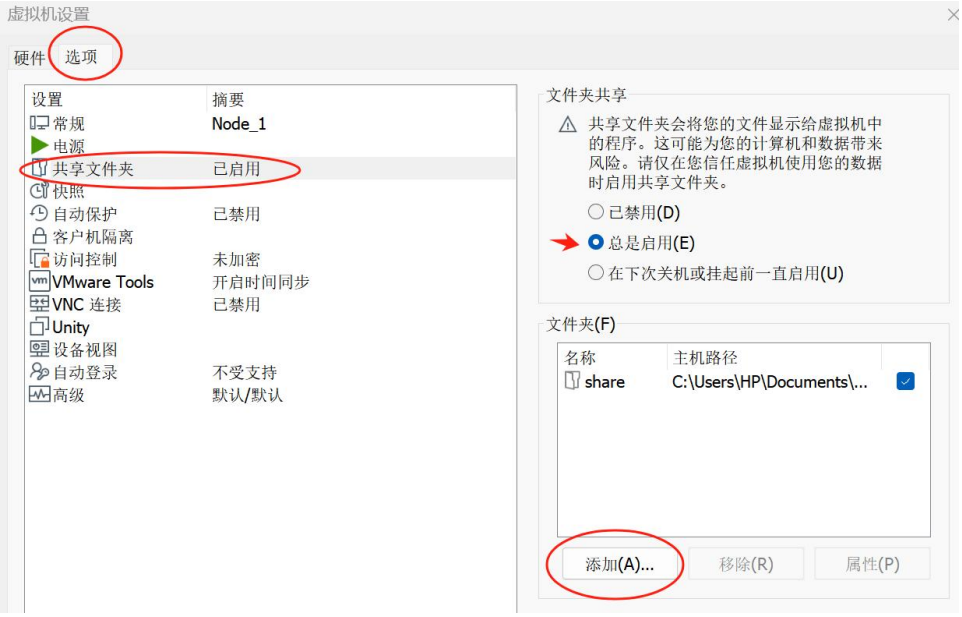


```
hadoop@ubuntu:~/Downloads/vmware-tools-distrib$ ll
total 408
drwxr-xr-x  9 hadoop hadoop   4096 Oct  1  2019 ./
drwxr-xr-x  3 hadoop hadoop   4096 May  9 02:38 ../
drwxr-xr-x  2 hadoop hadoop   4096 Oct  1  2019 bin/
drwxr-xr-x  5 hadoop hadoop   4096 Oct  1  2019 caf/
drwxr-xr-x  2 hadoop hadoop   4096 Oct  1  2019 doc/
drwxr-xr-x  5 hadoop hadoop   4096 Oct  1  2019 etc/
-rw-r--r--  1 hadoop hadoop 146996 Oct  1  2019 FILES
-rw-r--r--  1 hadoop hadoop   2538 Oct  1  2019 INSTALL
drwxr-xr-x  2 hadoop hadoop   4096 Oct  1  2019 installer/
drwxr-xr-x 14 hadoop hadoop   4096 Oct  1  2019 lib/
drwxr-xr-x  3 hadoop hadoop   4096 Oct  1  2019 vgauth/
-rwxr-xr-x  1 hadoop hadoop 227024 Oct  1  2019 vmware-install.pl*
```

安装过程根据提示选择 yes 或 no 选项，一般直接一路回车使用默认选项即可。（注：如果之前已经安装过 vmware-tools 则在询问是否需要覆盖安装文件时，需要针对性地做选择。）安装完成增强工具之后可以重启下虚拟机系统。然后在系统设置（System Settings）中找到 Displays 进行分辨率调整，如下图所示（记得点击 Apply）：



然后在虚拟机设置->选项中找到共享文件夹选项，添加主机 windows 文件夹，用于主机与虚拟机文件共享：



安装完成增强工具之后，主机与虚拟机之间可以共享剪贴板，方便文本复制。

3.2 熟悉常用的 Linux 操作

1）cd 命令：切换目录

（1） 切换到目录“/usr/local”

```
$ cd /usr/local
```

（2） 切换到当前目录的上一级目录

```
$ cd ..
```

（3） 切换到当前登录 Linux 系统的用户的自己的主文件夹

```
$ cd ~
```

2）ls 命令：查看文件与目录

查看目录“/usr”下的所有文件和目录

```
$ cd /usr
$ ls -al
```

3）mkdir 命令：新建目录

（1）进入“/tmp”目录，创建一个名为“a”的目录，并查看“/tmp”目录下已经存在哪些目录

```
$ cd /tmp
$ mkdir a
```

```
$ ls -al
```

（2）进入“/tmp”目录，创建目录“a1/a2/a3/a4”

```
$ cd /tmp
$ mkdir -p a1/a2/a3/a4
```

4）rmdir 命令：删除空的目录

（1）将上面创建的目录 a（在“/tmp”目录下面）删除

```
$ cd /tmp
$ rmdir a
```

（2）删除上面创建的目录“a1/a2/a3/a4” （在“/tmp”目录下面），然后查看“/tmp”目录下面存在哪些目录

```
$ cd /tmp
$ rmdir -p a1/a2/a3/a4
$ ls -al
```

5）cp 命令：复制文件或目录

（1）将当前用户的主文件夹下的文件.bashrc 复制到目录“/usr”下，并重命名为 bashrc1

```
$ sudo cp ~/.bashrc /usr/bashrc1
```

（2）在目录“/tmp”下新建目录 test，再把这个目录复制到“/usr”目录下

```
$ cd /tmp
$ mkdir test
$ sudo cp -r /tmp/test /usr
```

6）mv 命令：移动文件与目录，或更名

（1）将“/usr”目录下的文件 bashrc1 移动到“/usr/test”目录下

```
$ sudo mv /usr/bashrc1 /usr/test
```

（2）将“/usr”目录下的 test 目录重命名为 test2

```
$ sudo mv /usr/test /usr/test2
```

7）rm 命令：移除文件或目录

（1）将“/usr/test2”目录下的 bashrc1 文件删除

```
$ sudo rm /usr/test2/bashrc1
```

（2）将“/usr”目录下的 test2 目录删除

```
$ sudo rm -r /usr/test2
```

8）cat 命令：查看文件内容

查看当前用户主文件夹下的.bashrc 文件内容

```
$ cat ~/.bashrc
```

9）tac 命令：反向查看文件内容

反向查看当前用户主文件夹下的.bashrc 文件的内容

```
$ tac ~/.bashrc
```

10）more 命令：一页一页翻动查看

翻页查看当前用户主文件夹下的.bashrc 文件的内容

```
$ more ~/.bashrc
```

11）head 命令：取出前面几行

（1）查看当前用户主文件夹下.bashrc 文件内容前 20 行

```
$ head -n 20 ~/.bashrc
```

（2）查看当前用户主文件夹下.bashrc 文件内容，后面 50 行不显示，只显示前面几行

```
$ head -n -50 ~/.bashrc
```

12）tail 命令：取出后面几行

（1）查看当前用户主文件夹下.bashrc 文件内容最后 20 行

```
$ tail -n 20 ~/.bashrc
```

（2）查看当前用户主文件夹下.bashrc 文件内容，并且只列出 50 行以后的数据

```
$ tail -n +50 ~/.bashrc
```

13）touch 命令：修改文件时间或创建新文件

（1）在“/tmp”目录下创建一个空文件 hello，并查看文件时间

```
$ cd /tmp
$ touch hello
$ ls -l hello
```

（2）修改 hello 文件，将文件时间整为 5 天前

```
$ touch -d “5 days ago” hello
```

14）chown 命令：修改文件所有者权限

将 hello 文件所有者改为 root 帐号，并查看属性

```
$ sudo chown root /tmp/hello
$ ls -l /tmp/hello
```

15）chmod 命令：修改文件或目录的访问权限。

权限可以针对三类用户进行设置：文件所有者（u）、文件所属组（g）、其他用户（o），以及对所有用户（a）。

权限本身包括读（r）、写（w）、执行（x）三种。

4：可读权限（read）

2：可写权限（write）

1：可执行权限（execute）

```
$chmod a+x filename    #给所有用户添加执行权限
$chmod g-w filename    #移除组用户的写权限
$chmod 644 filename    #设置文件的所有者有读写权限，组用户和其他用户只有读权限
```

16）find 命令：文件查找

找出主文件夹下文件名为.bashrc 的文件

```
$ find ~ -name .bashrc
```

17）tar 命令：压缩命令

（1）在根目录“/”下新建文件夹 test，然后在根目录“/”下打包成 test.tar.gz

```
$ sudo mkdir /test
$ sudo tar -zcv -f /test.tar.gz test
```

（2）把上面的 test.tar.gz 压缩包，解压缩到“/tmp”目录

```
$ sudo tar -zxv -f /test.tar.gz -C /tmp
```

18）grep 命令：查找字符串

从“～/.bashrc”文件中查找字符串'examples'。此处，-n 表示需显示匹配所在行号。

```
$ grep -n 'examples' ~/.bashrc
```

19）配置环境变量

（1）请在“～/.bashrc”中设置，配置 Java 环境变量

首先，使用 vim 编辑器打开文件“~/.bashrc”，命令如下：

```
$ vim ~/.bashrc
```

然后，在该文件的最上面加入一行如下形式的语句：

```
export JAVA_HOME=JDK 安装路径
```

最后，执行如下命令使得环境变量配置生效：

```
$ source ~/.bashrc
```

（2）查看 JAVA_HOME 变量的值

```
$ echo $JAVA_HOME
```

3.3 进行 Hadoop 安装

3.3.1 创建 hadoop 用户

如果安装 Ubuntu 的时候不是用的 "hadoop" 用户，那么需要增加一个名为 hadoop 的用户。首先按 ctrl+alt+t 打开终端窗口，输入如下命令创建新用户：

```
sudo useradd -m hadoop -s /bin/bash
```

这条命令创建了可以登陆的 hadoop 用户，并使用 /bin/bash 作为 shell。输入一下命令设置用户密码，按提示输入两次密码：

```
sudo passwd hadoop
```

为 hadoop 用户增加管理员权限，方便部署：

```
sudo adduser hadoop sudo
```

最后注销当前用户（点击屏幕右上角的齿轮，选择注销），返回登陆界面。在登陆界面中选择刚创建的 hadoop 用户进行登陆。后续的安装配置都是在 hadoop 用户下进行的。

3.3.2 基础环境配置

(1) 更新 apt

用 hadoop 用户登录后，我们先更新一下 apt，后续我们使用 apt 安装软件，如果没更新可能有一些软件安装不了。按 ctrl+alt+t 打开终端窗口，执行如下命令：

```
sudo apt-get update
```

从软件源下载某些软件的过程中，可能由于网络方面的原因出现没法下载的情况，那么建议更改软件源。

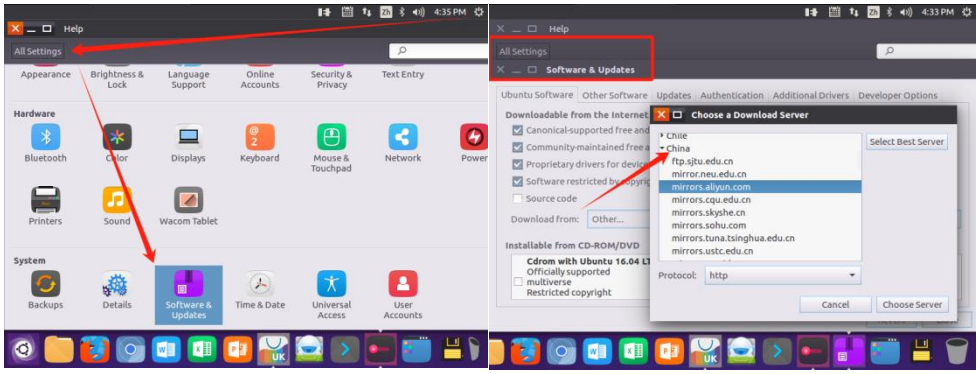
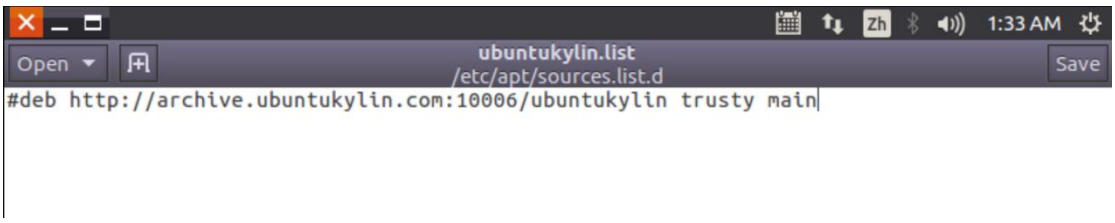


图 12. 选择软件源

如果更新软件源时出现如下错误：

```
hadoop@ubuntu:/usr/local/hadoop$ sudo apt-get update
Hit:1 http://mirrors.aliyun.com/ubuntu xenial InRelease
Hit:2 http://mirrors.aliyun.com/ubuntu xenial-updates InRelease
Hit:3 http://mirrors.aliyun.com/ubuntu xenial-backports InRelease
Hit:4 http://mirrors.aliyun.com/ubuntu xenial-security InRelease
Err:5 http://archive.ubuntukylin.com:10006/ubuntukylin trusty InRelease
  Could not connect to archive.ubuntukylin.com:10006 (124.126.103.228). - connec
t (111: Connection refused)
Reading package lists... Done
W: Failed to fetch http://archive.ubuntukylin.com:10006/ubuntukylin/dists/trusty
/InRelease  Could not connect to archive.ubuntukylin.com:10006 (124.126.103.228)
. - connect (111: Connection refused)
W: Some index files failed to download. They have been ignored, or old ones used
 instead.
```

可以将/etc/apt/sources.list.d/ubuntukylin.list 配置文件中的对应语句注释掉，再次执行 sudo apt-get update 即可。



(2) 安装 vim

后续需要更改一些配置文件，一般使用 vim 或者 gedit，通过如下命令安装 vim：

```
sudo apt-get install vim
```

3.3.3 安装 SSH、配置 SSH 无密码登陆

集群、单节点模式都需要用到 SSH 登陆（类似于远程登陆，你可以登录某台 Linux 主机，并且在上面运行命令），Ubuntu 默认已安装了 SSH **client**，此外还需要安装 SSH **server**：

```
sudo apt-get install openssh-server
```

安装后，可以使用如下命令登陆本机：

```
ssh localhost
```

此时会有如下提示(SSH 首次登陆提示)，输入 yes。然后按提示输入密码 hadoop，这样就登陆到本机了。但这样登陆是需要每次输入密码的，我们需要配置成 SSH 无密码登陆比较方便。首先退出刚才的 ssh，就回到了我们原先的终端窗口，然后利用 ssh-keygen 生成密钥，并将密钥加入到授权中：

```
exit
# 退出刚才的 ssh localhost

# 生成密钥：
ssh-keygen -t rsa   -C “your_name@pc_name”
# 会有提示，都按回车就可以
# 此处，用户名 yourname 及主机名 pc_name 用于标识，-C 为可选项

# 查看生成结果：
cd ~/.ssh/
cat ./id_rsa.pub >> ./authorized_keys   # 加入授权
```

1) 在 Linux 系统中，~ 代表的是用户的主文件夹，即 "/home/用户名" 这个目录，如你的用户名为 hadoop，则~就代表"/home/hadoop/"。

2) 此处 ssh-keygen 之前无需添加 sudo，否则生成的为 root 用户的.ssh。

以上设置完成后，再用 ssh localhost 命令，无需输入密码就可以**直接登陆**了。

3.3.4 安装 Java 环境

Hadoop3.3.5 需要 JDK 版本在 1.8 及以上。需要按照下面步骤来自己手动安装 JDK1.8。从 ftp 下载 jdk-8u371-linux-x64.tar.gz 文件到本地电脑，假设保存在

“/home/hadoop/Downloads/”目录下。在 Linux 命令行界面中，执行如下 Shell 命令（注意：当前登录用户名是 hadoop）：

```
cd /usr/lib
sudo mkdir jvm          #创建/usr/lib/jvm 目录用来存放 JDK 文件
cd ~                    #进入 hadoop 用户的主目录
cd Downloads            #注意区分大小写字母，刚才已经通过 FTP 软件把 JDK 安装包 jdk-8u371-linux-x64.tar.gz 上传到该目录下
sudo tar -zxvf ./jdk-8u371-linux-x64.tar.gz -C /usr/lib/jvm    #把 JDK 文件解压到/usr/lib/jvm 目录下
```

JDK 文件解压缩以后，可以执行如下命令到/usr/lib/jvm 目录查看一下：

```
cd ~
vim ~/.bashrc
```

上面命令使用 vim 编辑器打开了 hadoop 这个用户的环境变量配置文件（也可以使用图形化的 gedit 打开），请在这个文件的开头位置，添加如下几行内容：

```
export JAVA_HOME=/usr/lib/jvm/jdk1.8.0_371
export JRE_HOME=${JAVA_HOME}/jre
export CLASSPATH=.:${JAVA_HOME}/lib:${JRE_HOME}/lib
export PATH=${JAVA_HOME}/bin:$PATH
```

保存.bashrc 文件并退出 vim 编辑器。然后，继续执行如下命令让.bashrc 文件的配置立即生效：

```
source ~/.bashrc
```

使用如下命令查看是否安装成功：

```
java -version
```

如果能够在屏幕上返回如下信息，则说明安装成功：

```
hadoop@ubuntu:~$ java -version
java version "1.8.0_371"
Java(TM) SE Runtime Environment (build 1.8.0_371-b12)
Java HotSpot(TM) 64-Bit Server VM (build 25.162-b12, mixed mode)
```

至此，就成功安装了 Java 环境。下面就可以进入 Hadoop 的安装。

3.3.5 安装 Hadoop3.3.5

Hadoop 安装文件，可以到 Hadoop 官下载 hadoop-3.3.5.tar.gz，也可以直接从 ftp 下载。我们选择将 Hadoop 安装至 **/usr/local/** 中：

```
sudo tar -zxvf ~/下载/hadoop-3.3.5.tar.gz -C /usr/local     # 解压到/usr/local 中
cd /usr/local/
sudo mv ./hadoop-3.3.5/ ./hadoop                            # 将文件夹名改为 hadoop
sudo chown -R hadoop:hadoop ./hadoop                        # 修改文件权限
```

usr 是“Unix System Resources”的缩写。在 Linux 系统中，usr 目录位于根目录下，通常表示为/usr。这个目录主要用于存放用户安装的软件和文件，每个子目录都有自己的特定用途：

- /usr/bin：存放用户可执行的二进制文件，如常用的命令及工具。
- /usr/lib：存放共享库文件和一些系统级别的函数库。
- /usr/include：存放 C/C++等编程语言的头文件，用于开发过程中引用相关的函数和数据结构定义。
- /usr/local：存放**用户自行安装的软件和文件**，而不是通过操作系统默认的软件包管理系统进行安装的内容。
- /usr/share：存放共享的数据文件，如文档、帮助手册、示例配置文件等

Hadoop **解压后即可使用**。输入如下命令来检查 Hadoop 是否可用，成功则会显示 Hadoop 版本信息：

```
cd /usr/local/hadoop
./bin/hadoop version
```

相对路径与绝对路径: 请务必注意命令中的相对路径与绝对路径，本文后续出现的 ./bin/...，./etc/... 等包含 **./** 的路径，均为相对路径，以 /usr/local/hadoop 为当前目录。例如在 /usr/local/hadoop 目录中执行 ./bin/hadoop version 等同于执行 /usr/local/hadoop/bin/hadoop version。可以将相对路径改成绝对路径来执行，但**如果**你是在主文件夹~中执行 ./bin/hadoop version，执行的会是 /home/hadoop/bin/hadoop version，就**不是**我们所想要的了。

3.4 Hadoop 单机配置（非分布式）

Hadoop 默认模式为非分布式模式（本地模式），无需进行其他配置即可运行。非分布式即单 Java 进程，方便进行调试。我们可以执行例子来感受下 Hadoop 的运行。Hadoop 附带了丰富的例子。运行：

```
./bin/hadoop jar ./share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.5.jar
```

即可以看到所有例子，包括 wordcount、terasort、join、grep 等。在此我们选择运行 grep 例子，我们将 input 文件夹中的所有文件作为输入，筛选当中符合正则表达式 dfs[a-z.]+ 的单词并统计出现的次数，最后输出结果到 output 文件夹中。

```
cd /usr/local/hadoop

mkdir ./input

cp ./etc/hadoop/*.xml ./input                    # 将配置文件作为输入文件

./bin/hadoop jar ./share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.5.jar
grep ./input ./output 'dfs[a-z.]+'

cat ./output/*                                   # 查看运行结果
```

执行成功后将输出作业的相关信息，输出的结果是符合正则的单词 dfsadmin 出现了 1 次。

```
Terminal File Edit View Search Terminal Help                    6:55 AM
                Input split bytes=120
                Combine input records=0
                Combine output records=0
                Reduce input groups=1
                Reduce shuffle bytes=25
                Reduce input records=1
                Reduce output records=1
                Spilled Records=2
                Shuffled Maps =1
                Failed Shuffles=0
                Merged Map outputs=1
                GC time elapsed (ms)=0
                Total committed heap usage (bytes)=542113792
        Shuffle Errors
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=123
        File Output Format Counters
                Bytes Written=23
hadoop@ubuntu:/usr/local/hadoop$ ls output
part-r-00000  _SUCCESS
hadoop@ubuntu:/usr/local/hadoop$ ll output
total 20
drwxr-xr-x  2 hadoop hadoop 4096 May  9 06:54 ./
drwxr-xr-x 13 hadoop hadoop 4096 May  9 06:54 ../
-rw-r--r--  1 hadoop hadoop   11 May  9 06:54 part-r-00000
-rw-r--r--  1 hadoop hadoop   12 May  9 06:54 .part-r-00000.crc
-rw-r--r--  1 hadoop hadoop    0 May  9 06:54 _SUCCESS
-rw-r--r--  1 hadoop hadoop    8 May  9 06:54 ._SUCCESS.crc
hadoop@ubuntu:/usr/local/hadoop$ cat output/part-r-00000
1       dfsadmin
hadoop@ubuntu:/usr/local/hadoop$ cat output/*
1       dfsadmin
```

注意，Hadoop 默认不会覆盖结果文件，因此再次运行上面实例会提示出错，需要先将 ./output 删除。

```
rm -r ./output
```

3.5 Hadoop 伪分布式配置

Hadoop 可以在单节点上以伪分布式的方式运行，Hadoop 进程以**分离的 Java 进程**来运行，节点既作为 NameNode 也作为 DataNode，同时，读取的是 HDFS 中的文件。

Hadoop 的配置文件位于/usr/local/hadoop/etc/hadoop/中，伪分布式需要修改 2 个配置文件 core-site.xml 和 hdfs-site.xml。Hadoop 的配置文件是 xml 格式，每个配置以声明 property 的 name 和 value 的方式来实现。

修改配置文件 **core-site.xml**（通过 gedit 编辑会比较方便：gedit ./etc/hadoop/core-site.xml)，将当中的

```
<configuration>
</configuration>
```

修改为下面配置：

```
<configuration>
    <property>
        <name>hadoop.tmp.dir</name>
        <value>file:/usr/local/hadoop/tmp</value>
        <description>Abase for other temporary directories.</description>
    </property>
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://localhost:9000</value>
    </property>
</configuration>
```

1) 定义：在 Hadoop 配置中，fs.defaultFS 参数设置的 hdfs://localhost:9000 中的端口号 9000，作用是指定 NameNode RPC 通信的监听端口。
2) 作用：处理元数据操作请求，是 Hadoop 内部及客户端与 NameNode 通信的核心入口。
3) 使用者：hdfs 命令、DataNode、MapReduce 任务、Spark、Hive 等。
4) 区分：Web UI 端口为 9870，使用 HTTP 协议，提供可视化监控界面，供管理员查看集群状态、文件系统目录、日志等。使用者通常为管理员、运维人员。

同样的，修改配置文件 **hdfs-site.xml**：

```
<configuration>
    <property>
        <name>dfs.replication</name>
        <value>1</value>
    </property>
    <property>
        <name>dfs.namenode.name.dir</name>
        <value>file:/usr/local/hadoop/tmp/dfs/name</value>
    </property>
    <property>
        <name>dfs.datanode.data.dir</name>
        <value>file:/usr/local/hadoop/tmp/dfs/data</value>
    </property>
</configuration>
```

1) 这两个配置项分别定义了 HDFS 中 NameNode 和 DataNode 在本地文件系统上存储持久化数据的目录。它们是 HDFS 能够正常工作并保证数据不丢失的物理基础。
2) dfs.namenode.name.dir：指定 NameNode 在本地磁盘上存储 HDFS 命名空间的元数据的目录，用于存储 FsImage、EditLog。

3) 其中，fsimage 文件为 HDFS 命名空间的完整快照（目录树、文件属性等），edits 日志文件则用于记录自上次快照以来所有对元数据的修改操作（如创建/删除文件、修改权限等）。

Hadoop 配置文件说明:

Hadoop 的运行方式是由配置文件决定的（运行 Hadoop 时会读取配置文件），因此如果需要从伪分布式模式切换回非分布式模式，需要删除 core-site.xml 中的配置项。此外，伪分布式虽然只需要配置 fs.defaultFS 和 dfs.replication 就可以运行（官方教程如此），不过若没有配置 hadoop.tmp.dir 参数，则默认使用的临时目录为 /tmp/hadoop-${USERNAME}，对于本实验即/tmp/hadoop-hadoop，而这个目录在重启时有可能被系统清理掉，导致必须重新执行 format 才行。所以我们进行了设置，同时也指定 dfs.namenode.name.dir 和 dfs.datanode.data.dir，否则在接下来的步骤中可能会出错。

配置完成后，执行 NameNode 的**格式化**:

```
cd /usr/local/hadoop
./bin/hdfs namenode -format
```

成功的话，会看到 "successfully formatted" 的提示，具体返回信息类似如下：

```
2023-07-11 14:28:30,560 INFO namenode.NameNode: STARTUP_MSG:
/************************************************************


STARTUP_MSG: Starting NameNode
STARTUP_MSG:     host = hadoop/127.0.1.1
STARTUP_MSG:     args = [-format]
STARTUP_MSG:   version = 3.3.5
************************************************************/


......
2023-07-11 15:31:35,677 INFO common.Storage: Storage directory /usr/local/hadoop/tmp/dfs/name
has been successfully formatted.
2023-07-11 15:31:35,700 INFO namenode.FSImageFormatProtobuf: Saving image file
/usr/local/hadoop/tmp/dfs/name/current/fsimage.ckpt_0000000000000000000 using no compression
2023-07-11 15:31:35,770 INFO namenode.FSImageFormatProtobuf: Image file
/usr/local/hadoop/tmp/dfs/name/current/fsimage.ckpt_0000000000000000000 of size 393 bytes saved
in 0 seconds .
```

```
2023-07-11 15:31:35,810 INFO namenode.NNStorageRetentionManager: Going to retain 1 images
with txid >= 0
2023-07-11 15:31:35,816 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid = 0 when
meet shutdown.
2023-07-11 15:31:35,816 INFO namenode.NameNode: SHUTDOWN_MSG:
/************************************************************
SHUTDOWN_MSG: Shutting down NameNode at hadoop/127.0.1.1

************************************************************/
```

接着**开启** NameNode 和 DataNode **守护进程**。

```
cd /usr/local/hadoop
./sbin/start-dfs.sh          #start-dfs.sh 是个完整的可执行文件，中间没有空格
```

若提示 Error: JAVA_HOME is not set and could not be found. 的错误，**可能**是之前设置 JAVA_HOME 环境变量没有设置好，通过 echo $JAVA_HOME 指令来查看 JAVA_HOME 的配置情况，如果不正确请按上面的教程先设置好 JAVA_HOME 变量。

```
hadoop@ubuntu:/usr/local/hadoop$ echo $JAVA_HOME
/usr/lib/jvm/jdk1.8.0_371
hadoop@ubuntu:/usr/local/hadoop$ ./sbin/start-dfs.sh
Starting namenodes on [localhost]
localhost: ERROR: JAVA_HOME is not set and could not be found.
Starting datanodes
localhost: ERROR: JAVA_HOME is not set and could not be found.
Starting secondary namenodes [liveupdate5.f5.lenovo.com.cn]
liveupdate5.f5.lenovo.com.cn: ERROR: JAVA_HOME is not set and could not be found.
```

如果已经按照前面教程在.bashrc 文件中设置了 JAVA_HOME，**还是出现** Error: JAVA_HOME is not set and could not be found. 的错误，那么，请到 hadoop 的安装目录修改配置文件“/usr/local/hadoop/etc/hadoop/**hadoop-env.sh**”，在里面**找到**“export JAVA_HOME=${JAVA_HOME}”这行，然后，把它**修改**成本机 JAVA 安装路径的具体地址，比如，“export JAVA_HOME=/usr/lib/jvm/jdk1.8.0_371”，然后，再次启动 Hadoop。

```
File Edit View Search Tools Documents Help                    8:23 AM
Open                                                          Save
45 ###
46
47 # Technically, the only required environment variable is JAVA_HOME.
48 # All others are optional.  However, the defaults are probably not
49 # preferred.  Many sites configure these options outside of Hadoop,
50 # such as in /etc/profile.d
51
52 # The java implementation to use. By default, this environment
53 # variable is REQUIRED on ALL platforms except OS X!
54 # export JAVA_HOME=
55 export JAVA_HOME=/usr/lib/jvm/jdk1.8.0_371
56
57 # Location of Hadoop.  By default, Hadoop will attempt to determine
58 # this location based upon its execution path.
59 # export HADOOP_HOME=
60
61 # Location of Hadoop's configuration information.  i.e., where this
62 # file is living. If this is not defined, Hadoop will attempt to
63 # locate it based upon its execution path.
64 #
```

```
hadoop@ubuntu:/usr/local/hadoop$ echo $JAVA_HOME
/usr/lib/jvm/jdk1.8.0_371
hadoop@ubuntu:/usr/local/hadoop$ ./sbin/start-dfs.sh
Starting namenodes on [localhost]
localhost: ERROR: JAVA_HOME is not set and could not be found.
Starting datanodes
localhost: ERROR: JAVA_HOME is not set and could not be found.
Starting secondary namenodes [liveupdate5.f5.lenovo.com.cn]
liveupdate5.f5.lenovo.com.cn: ERROR: JAVA_HOME is not set and could not be found.
hadoop@ubuntu:/usr/local/hadoop$ gedit etc/hadoop/
capacity-scheduler.xml            httpfs-env.sh                     mapred-site.xml
configuration.xsl                 httpfs-log4j.properties           shellprofile.d/
container-executor.cfg            httpfs-site.xml                   ssl-client.xml.example
core-site.xml                     kms-acls.xml                      ssl-server.xml.example
hadoop-env.cmd                    kms-env.sh                        user_ec_policies.xml.template
hadoop-env.sh                     kms-log4j.properties              workers
hadoop-metrics2.properties        kms-site.xml                      yarn-env.cmd
hadoop-policy.xml                 log4j.properties                  yarn-env.sh
hadoop-user-functions.sh.example  mapred-env.cmd                    yarnservice-log4j.properties
hdfs-rbf-site.xml                 mapred-env.sh                     yarn-site.xml
hdfs-site.xml                     mapred-queues.xml.template
hadoop@ubuntu:/usr/local/hadoop$ gedit etc/hadoop/hadoop-env.sh
hadoop@ubuntu:/usr/local/hadoop$ ./sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [liveupdate5.f5.lenovo.com.cn]
```

启动过程中会有各种信息输出，若出现如下 SSH 提示，输入 yes 即可。

```
Starting namenodes on [localhost]
localhost: starting namenode, logging to /usr/local/hadoop/logs/hadoop-hadoop-n
localhost: starting datanode, logging to /usr/local/hadoop/logs/hadoop-hadoop-d
Starting secondary namenodes [0.0.0.0]
The authenticity of host '0.0.0.0 (0.0.0.0)' can't be established.
ECDSA key fingerprint is a9:28:e0:4e:89:40:a4:cd:75:8f:0b:8b:57:79:67:86.
Are you sure you want to continue connecting (yes/no)? yes
```

如果启动 Hadoop 时遇到输出非常多“ssh: Could not resolve hostname xxx”的异常情况，如下图所示：

```
hadoop@vm: /usr/local/hadoop
library: ssh: Could not resolve hostname library: Name or servi
ce not known
which: ssh: Could not resolve hostname which: Name or service n
ot known
disabled: ssh: Could not resolve hostname disabled: Name or ser
vice not known
warning:: ssh: Could not resolve hostname warning:: Name or ser
vice not known
stack: ssh: Could not resolve hostname stack: Name or service n
ot known
```

这个并不是 ssh 的问题，可通过设置 Hadoop 环境变量来解决。首先按键盘的 ctrl + c 中断启动，然后在~/.bashrc 中，增加如下两行内容（设置过程与 JAVA_HOME 变量一样，其中 **HADOOP_HOME** 为 Hadoop 的安装目录）：

```
export HADOOP_HOME=/usr/local/hadoop
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
```

保存后，务必执行 source ~/.bashrc 使变量设置生效，然后再次执行 ./sbin/start-dfs.sh 启动 Hadoop。

启动完成后，可以通过命令 jps 来判断是否成功启动，若成功启动则会列出如下进程: "NameNode"、"DataNode" 和 "SecondaryNameNode"

- 如果 SecondaryNameNode 没有启动，请运行 sbin/stop-dfs.sh 关闭进程，然后再次尝试启动尝试。
- 如果没有 NameNode 或 DataNode ，那就是配置不成功，请仔细检查之前步骤，或通过查看启动日志排查原因。

```
hadoop@ubuntu:/usr/local/hadoop$ jps
8560 SecondaryNameNode
8210 NameNode
8756 Jps
8334 DataNode
```

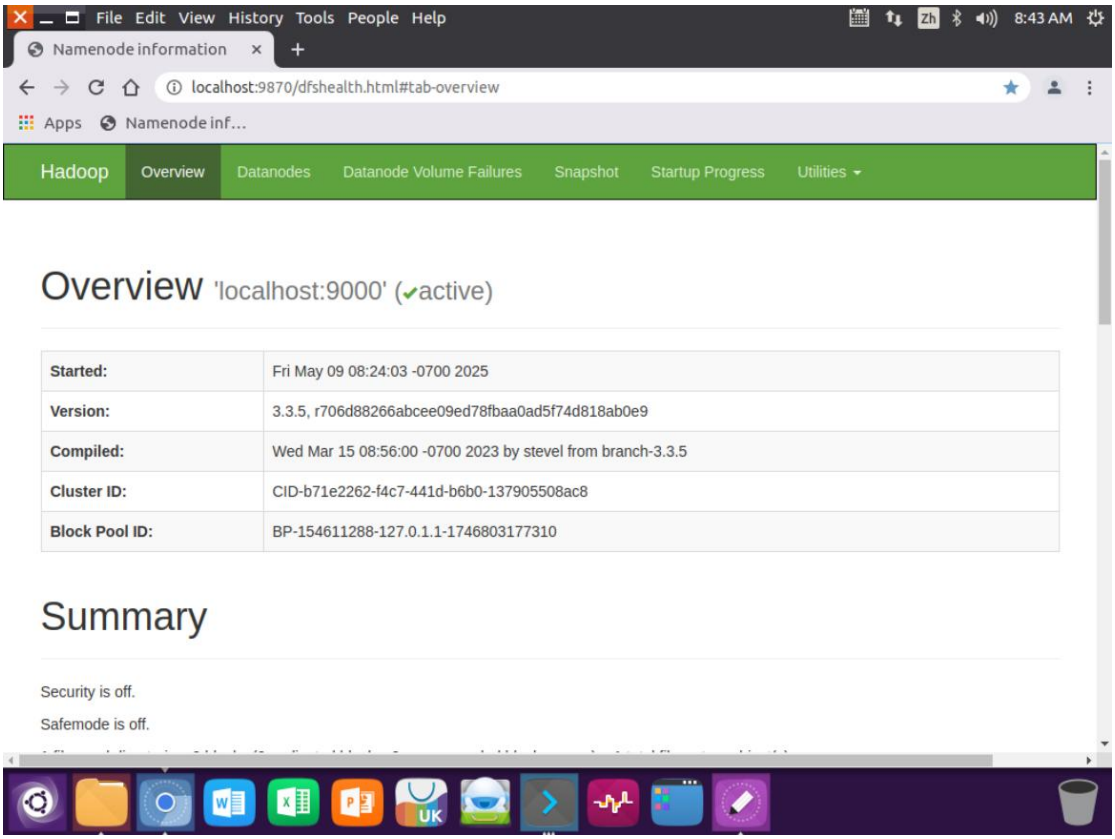
Hadoop 无法正常启动的解决方法: 一般可以查看启动日志来排查原因，注意几点：

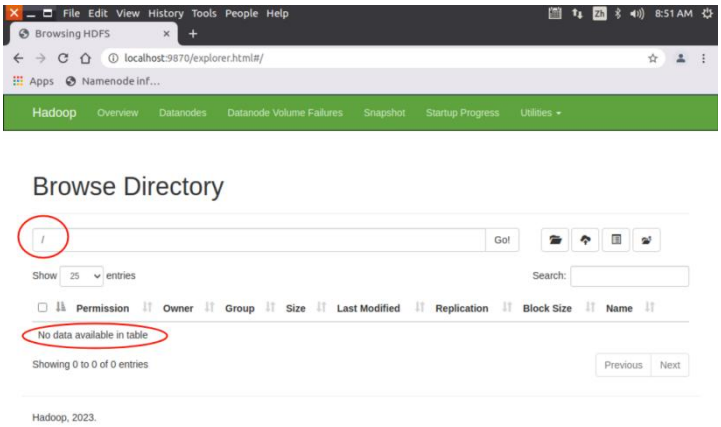
- 启动日志信息在 /usr/local/hadoop/logs/hadoop-hadoop-namenode-ubuntu.log 中，所以应该查看这个后缀为 **.log** 的文件；
- 每一次的启动日志都是追加在日志文件之后，所以得拉到最后面看，对比下记录的时间就知道了。
- 一般出错的提示在最后面，通常是写着 Fatal、Error、Warning 或者 Java Exception 的地方。
- 可以将 log 上下文输入 deepseek 进行询问，定位问题；或者在网上搜索一下出错信息，看能否找到一些相关的解决方法。一般而言，易出错的问题，网上会有很多案例，如果检索不到相关信息，很可能是自己前面的配置问题，如空格或者单词拼写错误等，需反复排查确认。

此外，若是 DataNode 没有启动，可尝试如下的方法（注意这会删除 HDFS 中原有的所有数据，如果原有的数据很重要请不要这样做）：

```
# 针对 DataNode 没法启动的解决方法
cd /usr/local/hadoop
./sbin/stop-dfs.sh              # 关闭
rm -r ./tmp                     # 删除 tmp 文件，注意这会删除 HDFS 中原有的所有数据
./bin/hdfs namenode -format     # 重新格式化 NameNode
./sbin/start-dfs.sh             # 重启
```

成功启动后，可以访问 **Web 界面** http://localhost:9870 查看 NameNode 和 Datanode 信息，还可以在线查看 HDFS 中的文件。





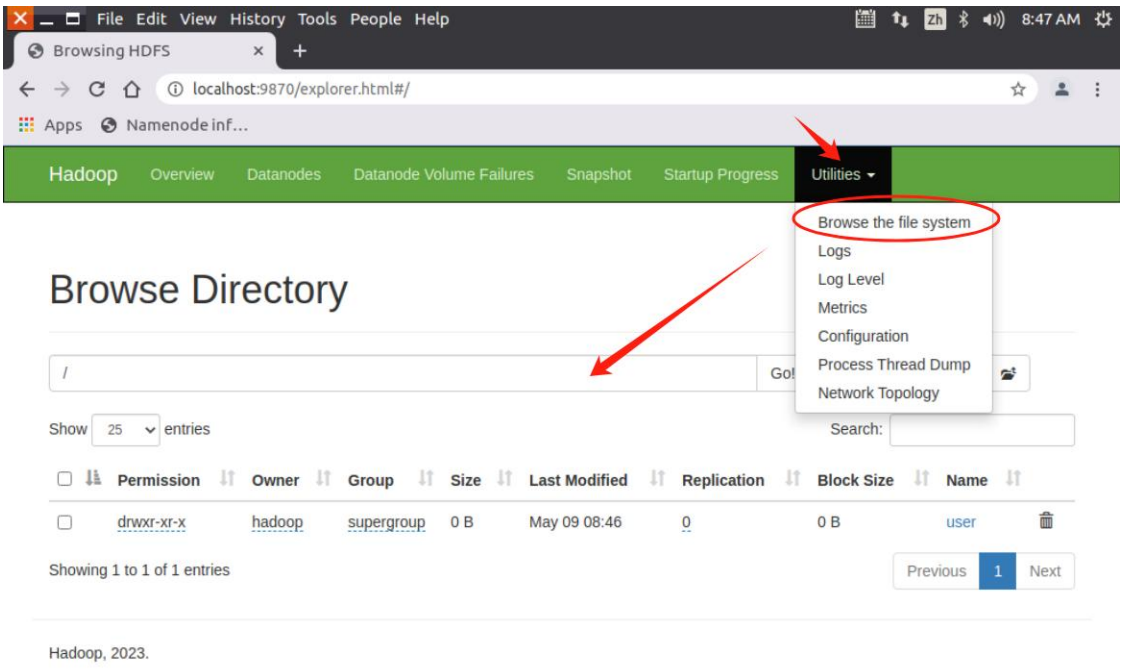
3.6 运行 Hadoop 伪分布式实例

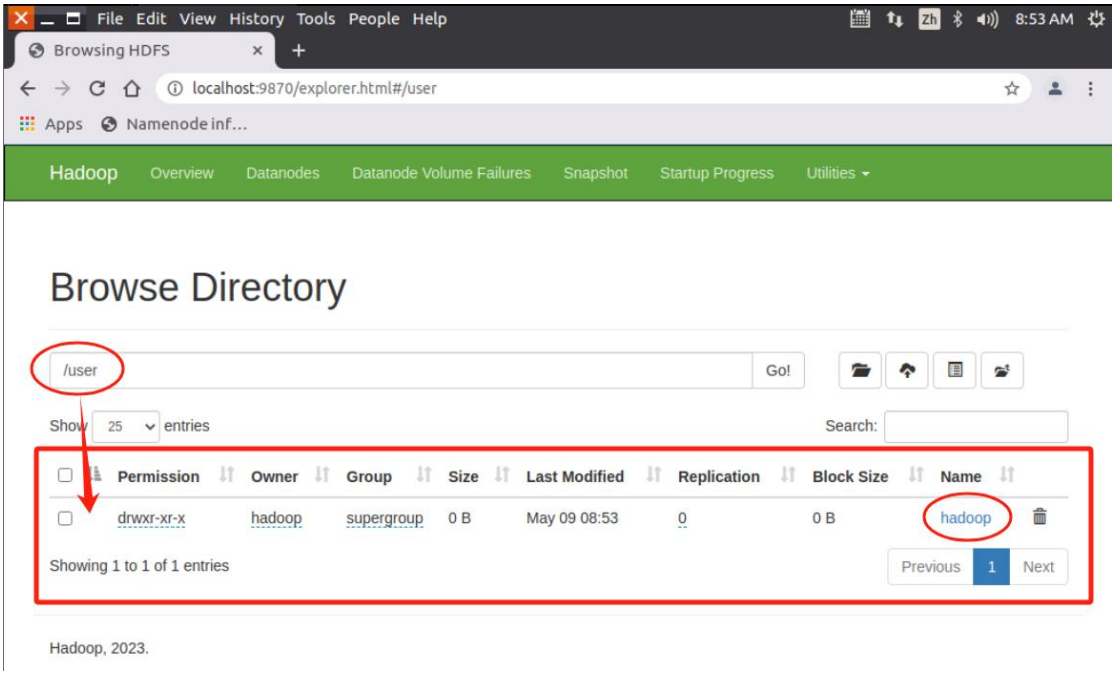
上面的单机模式，grep 例子读取的是本地数据，伪分布式读取的则是 HDFS 上的数据。要使用 HDFS，首先需要在 HDFS 中创建用户目录：

需要注意的是，Hadoop 系统安装好以后，**第一次使用 HDFS 时**，需要首先在 HDFS 中**创建用户目录**。本教程全部采用 hadoop 用户登录 Linux 系统，因此，

需要在 HDFS 中（hdfs://localhost:9000/user/hadoop）为 hadoop 用户创建一个用户目录，命令如下：

```
./bin/hdfs dfs -mkdir -p /user/hadoop
```





接着将 ./etc/hadoop 中的 .xml 文件作为输入文件复制到分布式文件系统中，即将 /usr/local/hadoop/etc/hadoop 复制到分布式文件系统中的

/user/hadoop/input 中。我们使用的是 hadoop 用户，并且已创建相应的用户目录 /user/hadoop ，因此在命令中就可以使用相对路径如 input，其对应的绝对路径就是 /user/hadoop/input:

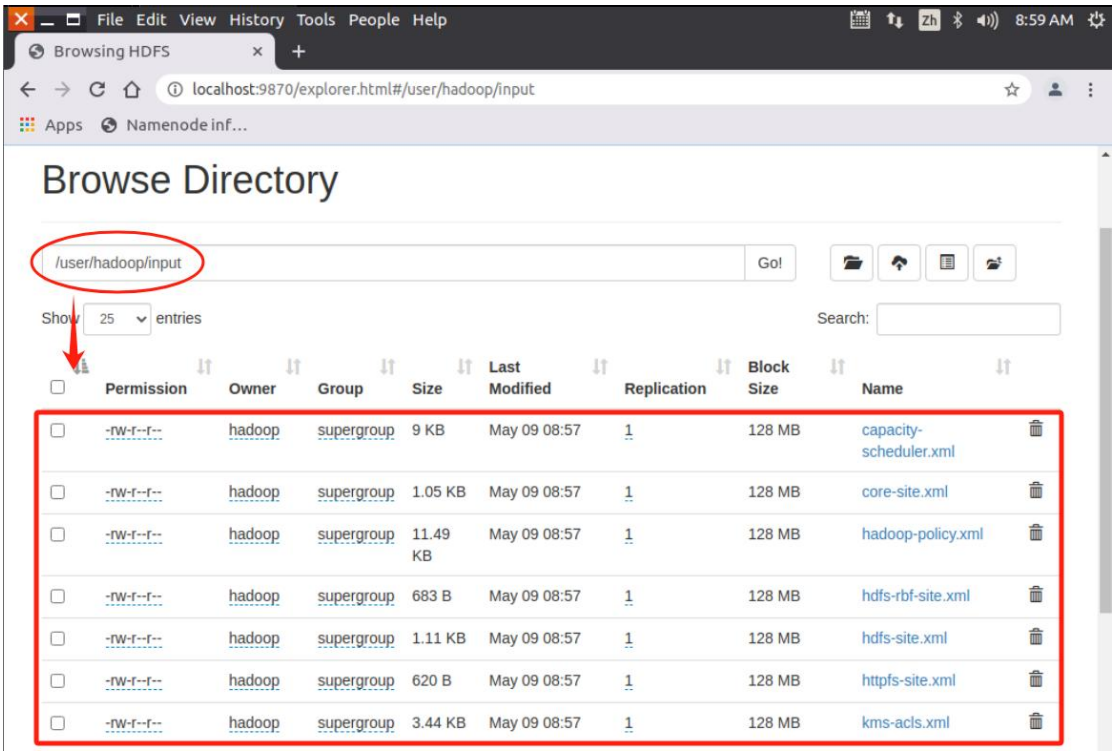
```
./bin/hdfs dfs -mkdir input
./bin/hdfs dfs -put ./etc/hadoop/*.xml input
```

复制完成后，可以通过如下命令查看文件列表：

```
./bin/hdfs dfs -ls input
```

```
hadoop@ubuntu:/usr/local/hadoop$ ./bin/hdfs dfs -mkdir -p /user/hadoop
hadoop@ubuntu:/usr/local/hadoop$ ./bin/hdfs dfs -mkdir input
hadoop@ubuntu:/usr/local/hadoop$ ./bin/hdfs dfs -put ./etc/hadoop/*.xml input
hadoop@ubuntu:/usr/local/hadoop$ ./bin/hdfs dfs -ls input
Found 10 items
-rw-r--r--   1 hadoop supergroup       9213 2025-05-09 08:57 input/capacity-scheduler.xml
-rw-r--r--   1 hadoop supergroup       1075 2025-05-09 08:57 input/core-site.xml
-rw-r--r--   1 hadoop supergroup      11765 2025-05-09 08:57 input/hadoop-policy.xml
-rw-r--r--   1 hadoop supergroup        683 2025-05-09 08:57 input/hdfs-rbf-site.xml
-rw-r--r--   1 hadoop supergroup       1133 2025-05-09 08:57 input/hdfs-site.xml
-rw-r--r--   1 hadoop supergroup        620 2025-05-09 08:57 input/httpfs-site.xml
-rw-r--r--   1 hadoop supergroup       3518 2025-05-09 08:57 input/kms-acls.xml
-rw-r--r--   1 hadoop supergroup        682 2025-05-09 08:57 input/kms-site.xml
-rw-r--r--   1 hadoop supergroup        758 2025-05-09 08:57 input/mapred-site.xml
-rw-r--r--   1 hadoop supergroup        690 2025-05-09 08:57 input/yarn-site.xml
```

从 web 管理界面中也可以看到相应文件信息：



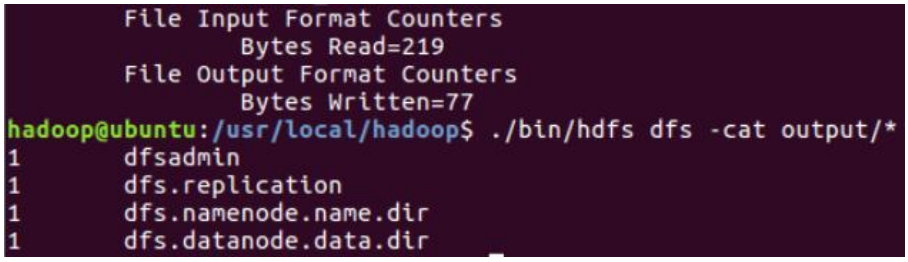
伪分布式运行 MapReduce 作业的方式跟单机模式相同，区别在于伪分布式读取的是 HDFS 中的文件（可以将单机步骤中创建的本地 input 文件夹，输出结果 output 文件夹都删掉来验证这一点）。

```
./bin/hadoop jar ./share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.5.jar grep input output 'dfs[a-z.]+'
```

查看运行结果的命令（查看的是位于 HDFS 中的输出结果）：

```
./bin/hdfs dfs -cat output/*
```

结果如下，注意到刚才我们已经更改了配置文件，所以运行结果不同（即下图中的后 3 个查找结果）。



我们也可以将运行结果取回到本地：

```
rm -r ./output                    # 先删除本地的 output 文件夹（如果存在）
./bin/hdfs dfs -get output ./output   # 将 HDFS 上的 output 文件夹拷贝到本机
cat ./output/*
```

Hadoop 运行程序时，输出目录不能预先存在，否则会提示错误 "org.apache.hadoop.mapred.FileAlreadyExistsException: Output directory hdfs://localhost:9000/user/hadoop/output already exists" ，因此若要再次执行，需要执行如下命令删除 output 文件夹:

```
./bin/hdfs dfs -rm -r output    # 删除 output 文件夹
```

若要关闭 Hadoop，则运行

```
./sbin/stop-dfs.sh
```

注意：下次启动 hadoop 时，无需进行 NameNode 的初始化，只需要运行 ./sbin/start-dfs.sh 就可以。

3.7 熟悉常用的 Hadoop 操作

（1）使用 hadoop 用户登录 Linux 系统，启动 Hadoop（Hadoop 的安装目录为“/usr/local/hadoop”），第一次使用 HDFS 时，需**为 hadoop 用户在 HDFS 中创建用户目录“/user/hadoop”**

```
$ cd /usr/local/hadoop
$ ./sbin/start-dfs.sh
$ ./bin/hdfs dfs -mkdir -p /user/hadoop
```

（2）接着在 HDFS 的目录“/user/hadoop”下，创建 test 文件夹，并查看文件列表

```
$ cd /usr/local/hadoop
$ ./bin/hdfs dfs -mkdir test
$ ./bin/hdfs dfs -ls .
```

（3）将 Linux 系统本地的“～/.bashrc”文件上传到 HDFS 的 test 文件夹中，并查看 test

```
$ cd /usr/local/hadoop
$ ./bin/hdfs dfs -put ~/.bashrc test
$ ./bin/hdfs dfs -ls test
```

（4）将 HDFS 文件夹 test 复制到 Linux 系统本地文件系统的“/usr/local/hadoop”目录下

```
$ cd /usr/local/hadoop
$./bin/hdfs dfs -get test ./
```

为了避免频繁输入 hadoop 命令的绝对路径，我们也可以将其添加到 PATH 环境变量中：

```
$ gedit ~/.bashrc
```

```
File Edit View Search Tools Documents Help                    9:20 AM
Open                                                          Save
92 alias la= ls -A
93 alias l='ls -CF'
94
95 # Add an "alert" alias for long running commands.  Use like so:
96 #   sleep 10; alert
97 alias alert='notify-send --urgency=low -i "$([ $? = 0 ] && echo terminal || echo error)" "$(history|tail -n1|
   sed -e '\''s/^\s*[0-9]\+\s*//;s/[;&|]\s*alert$//'\'')"'
98
99 # Alias definitions.
100 # You may want to put all your additions into a separate file like
101 # ~/.bash_aliases, instead of adding them here directly.
102 # See /usr/share/doc/bash-doc/examples in the bash-doc package.
103
104 if [ -f ~/.bash_aliases ]; then
105     . ~/.bash_aliases
106 fi
107
108 # enable programmable completion features (you don't need to enable
109 # this, if it's already enabled in /etc/bash.bashrc and /etc/profile
110 # sources /etc/bash.bashrc).
111 if ! shopt -oq posix; then
112   if [ -f /usr/share/bash-completion/bash_completion ]; then
113     . /usr/share/bash-completion/bash_completion
114   elif [ -f /etc/bash_completion ]; then
115     . /etc/bash_completion
116   fi
117 fi
118
119
120 export JAVA_HOME=/usr/lib/jvm/jdk1.8.0_371
121 export JRE_HOME=${JAVA_HOME}/jre
122 export CLASSPATH=.:${JAVA_HOME}/lib:${JRE_HOME}/lib
123 export PATH=${JAVA_HOME}/bin:$PATH
124
125 export HADOOP_HOME=/usr/local/hadoop
126 export PATH=${HADOOP_HOME}/bin:$PATH
127
```

在文件末尾添加如下信息：

export HADOOP_HOME=/usr/local/hadoop

export PATH=${HADOOP_HOME}/bin:$PATH

保存退出后，终端中执行 source ~/.bashrc 指令，使新的配置生效。此后，命令 /usr/local/hadoop/bin/hdfs dfs 可以简化为 hdfs dfs。

3.8 创建两台虚拟机配置并运行上述 Hadoop 分布式实例

该任务由同学们查找资料自主完成，重点体现如下内容：如何修改配置文件、如何监控系统资源消耗及作业执行情况。报告模板如下：

4 实验报告

《大数据技术原理与应用》实验报告				
题目：		姓名		日期
实验环境：				
实验内容与完成情况：				
出现的问题：				
解决方案（列出遇到的问题和解决办法，列出没有解决的问题）：				